

Q1.

- (a) What is risk projection in software project management? Briefly explain its purpose. **(2 Marks)**

Risk projection in software project management is the process of estimating the likelihood and potential impact of identified risks on a project's objectives. Its purpose is to prioritize risks based on their severity and probability, helping managers plan effective mitigation and contingency strategies to minimize project threats.

- (b) In a project, three risks have the following data - **risk name (Probability, Impact)**: R1 (0.2, \$40,000), R2 (0.5, \$25,000), R3 (0.1, \$90,000). Calculate the risk exposure (RE) for each risk and identify which risk should be given the highest priority. **(3 Marks)**

Risk Exposure (RE) is calculated using the formula:

$$RE = \text{Probability} \times \text{Impact}$$

Now, compute for each risk:

Risk	Probability	Impact (\$)	Risk Exposure (RE = P × I)
R1	0.2	40,000	$0.2 \times 40,000 = 8,000$
R2	0.5	25,000	$0.5 \times 25,000 = 12,500$
R3	0.1	90,000	$0.1 \times 90,000 = 9,000$

Highest Risk Exposure: R2 = \$12,500

Therefore, Risk R2 should be given the highest priority, as it has the greatest potential loss.

- (c) What is the difference between a catastrophic and critical risk category? **(1 Marks)**

A **catastrophic risk** is one that can cause complete project failure, severe financial loss, or endanger human safety—its impact is extremely high and often unacceptable.

In contrast, a **critical risk** causes major disruption or significant loss but does not completely destroy the project; recovery is possible with corrective action and additional resources.

Q2. You were supposed to explain the following types of couplings and cohesions: **(2 marks each)**

Coupling – data, stamp and common

Cohesion – functional

Q3. (2 marks describing + 0.5 Marks for diagram) per part.

(a) Speech Recognition System

Architectural Style: Blackboard Architecture

Explanation:

In the **Blackboard architecture**, multiple independent modules (called knowledge sources) work cooperatively by reading and updating a **shared data structure** known as the *blackboard*.

- Each module contributes partial solutions or intermediate results to the blackboard.
- Other modules use this shared information to refine or generate further results.
- The process continues until the goal (e.g., recognized speech) is achieved.

Hence, since the modules in the speech recognition system share a *common knowledge base* to store and retrieve intermediate results, it follows the **Blackboard architectural style**.

(b) Data Processing System

Architectural Style: Pipe and Filter Architecture

Explanation:

In the **Pipe and Filter architecture**, data flows through a series of **processing elements (filters)** connected by **data streams (pipes)**.

- Each filter performs a specific transformation on the input data.
- The output of one stage serves as the input to the next.

Therefore, since the system processes sensor data through sequential stages like filtering and aggregation, it follows the **Pipe and Filter architectural style**.

Q4. (3 Marks each)

(a) Top-Down Integration Approach:

In the **top-down integration approach**, testing starts from the **main control module** (top-level) and progresses downward through the control hierarchy. The upper-level modules are tested first, and the lower-level modules are integrated and tested step-by-step using **stubs** (temporary modules that simulate the behavior of subordinate modules not yet integrated).

Problems Associated with Top-Down Approach:

- **Stub dependency:** Requires creation of many stubs, increasing development and maintenance effort.
- **Late availability of lower modules:** Lower-level modules are tested late in the process.
- **Difficult interface testing early:** Some data interfaces and complex interactions may not be adequately tested until late stages.
- **Limited early visibility of actual functionality:** As stubs replace real modules, realistic system behavior isn't observed early.

(b) Resolution by Bottom-Up Approach:

Yes, the **bottom-up approach** resolves some of these issues.

In the **bottom-up integration approach**, testing starts from the **lowest-level modules** and proceeds upward. **Drivers** (temporary modules that simulate higher-level modules) are used to test lower modules until the upper modules are available.

How it resolves issues:

- **No stubs needed:** Replaces them with drivers, which are generally simpler to implement.
- **Early testing of real modules:** Lower-level functional modules are tested early, giving quicker feedback on core logic.
- **Interfaces tested early:** Since modules are combined from the bottom up, real interactions between modules are verified sooner.
- **System reliability improves early:** Foundational components are verified before integrating with higher-level control logic.